

**Baranya Vármegyei Szaképzési Centrum Komlói Technikum,
Szakképző Iskola és Kollégium**

Vizsgaremek

**Bázing Bence György
Püspöki János Ákos
Várhalmi Noel**

2026

Tartalomjegyzék

Fejlesztőcsapat.....	5
1.1. A projekt bemutatása.....	5
1.2. Célkitűzések.....	6
1.3. Főbb jellemzők.....	6
2. Rendszerterv és Architektúra.....	7
2.1. Technológiai Stack.....	7
2.2. Szerkezeti felépítés.....	7
3. Funkcionális Specifikáció.....	8
3.1. Kétpaneles Fájlkezelés.....	8
3.2. Hibrid Web-Integráció.....	8
3.3. Támogatott Platformok.....	8
4. Felhasználói Útmutató (Használat).....	9
4.1. A kezelőfelület felépítése.....	9
4.2. Fájlműveletek (Kontextusmenü).....	9
4.3. A hibrid web-funkció használata.....	10
4.4. Nyelvi beállítások és testreszabás.....	10
4.6. Hálózati kapcsolat (LAN / Mobil tárhely).....	11
4.7. Beépített Szövegszerkesztő.....	11
4.7.1 Funkciók és használat.....	11
4.8. Hardverkövetelmények.....	12
4.8.1 Tárolóegység (Storage).....	12
4.8.2 Memória (RAM).....	12
4.8.3 Processzor (CPU).....	12
4.9 Telepítés és Rendszerkövetelmények.....	13
4.9.1 Rendszerkövetelmények.....	13
4.9.2 Letöltési források.....	13
4.9.3 Telepítési folyamat.....	13
5. Fejlesztői dokumentáció (OntroSYS Core).....	14
5.1. Programindítási szekvencia (Bootstrapping).....	14
5.2. A grafikus felület inicializálása (screen.java).....	16
5.3. A GUI komponensek elhelyezése (GUI.java).....	16
5.4. Dinamikus menürendszer felépítése (menubar.java).....	17
5.5. Eseménykezelés és Interakciók (actionlisteners.java).....	18
5.5.1. Globális komponensek és Böngésző inicializálás.....	18
5.5.2. Menüsor funkciók.....	18
5.5.3. Szinkronizált görgetési mechanizmus (AdjustmentListener).....	18
5.5.4. Intelligens duplikált kattintás (Double Click) logika.....	19
5.6. Adatfeldolgozási logika (dataprocess.java).....	20
5.6.1. Meghajtókezelés (updatedrive).....	20
5.6.2. Panelstruktúra betöltése (loadleftsidedstructure / loadrightsidedstructure).....	20
5.6.3. Dinamikus újratöltés (reloadleftsidedstructure).....	21
5.6.4. Intelligens rendezés (sortbynameleft).....	21
5.7. Fájlrendszer-beolvasó modul (fileolvaso.java).....	22
5.7.1. Működési mechanizmus.....	22
5.7.2. Hibakezelés és Naplózás.....	22

5.8. Kontextusfüggő vezérlés (rightclickmenu.java).....	23
5.8.1. Dinamikus menüfelépítés (setuprightclickmenu).....	23
5.8.2. Akció-kódolás és Állapotkezelés.....	23
5.8.3. Fájlműveletek Implementációja.....	23
5.9. Fájlműveletek vizualizációja és aszinkron végrehajtása (fileactionGUI.java).....	24
5.9.1. Aszinkron végrehajtás (Multithreading).....	24
5.9.2. Haladási naplózás és Sebességmérés.....	25
5.9.3. Felhasználói kontroll (Pause/Stop).....	25
5.10. Fizikai másolási mechanizmus (FileCopier.java).....	26
5.10.1. Rekurzív könyvtármásolás.....	26
5.10.2. Intelligens fájlnev-ütközés kezelés.....	26
5.10.3. Folyamatvezérlés (Stop & Cancel).....	26
5.10.4. Sebességmérés és Bufferelés.....	27
5.11. Fájlmozgatási mechanizmus (FileCut.java).....	27
5.11.1. Atomi jellegű mozgatási logika.....	27
5.11.2. Biztonsági protokoll megszakítás (Cancel) esetén.....	28
5.11.3. Rekurzív forrástisztítás (deleteDirectory).....	28
5.12. Fájl törlési mechanizmus (Deletefile.java).....	29
5.12.1. Rekurzív törlési protokoll.....	29
5.12.2. Folyamatkövetés és Számlálás.....	29
5.12.3. Biztonságos megszakítás.....	29
5.13. Képnézegető modul (Kísérleti fázis - imageviewer.java).....	30
5.13.1. Interaktív képmozgatás (huzogato).....	30
5.13.2. Dinamikus skálázás (Zoom).....	30
5.13.3. Valós idejű renderelési ciklus.....	30
5.14. Dinamikus nyelvkezelési logika (language_change.java).....	31
5.14.1. Útvonal-átirányítási mechanizmus.....	31
5.14.2. Adatfolyam-vezérlés.....	31
5.15. Beépített Szövegszerkesztő Modul (TextEditorv2.java).....	32
5.15.1. Fájlkezelés és I/O műveletek.....	32
5.15.2. Dinamikus Vizuális Beállítások.....	32
5.15.3. Speciális Fájlműveletek: Átnevezés.....	32
5.16. Fejlesztői Terminál és Rendszernapló (Terminal.java).....	33
5.16.1. Központosított Naplózás (Static Log).....	33
5.16.2. Parancskezelő és Input Interfész.....	33
5.17. Integrált Webes Motor (OntroSYS_WEBTEST).....	34
5.17.1. Windowless Rendering és Megjelenítés.....	34
5.17.2. Hibrid Görgetési Mechanizmus (JS Injection).....	34
5.17.3. Fókuszkezelési Logika.....	34
5.17.4. Intelligens URL Kezelés.....	35
5.18. Globális Adatkezelés és Rendszerállapot (Gvar.java).....	35
5.18.1. GUI és Elrendezés Referenciák.....	35
5.18.2. Kétpaneles Adatstruktúra.....	35
5.18.3. Állapotmegőrzés (Navigation State).....	35
5.18.4. Konfigurációs és Hálózati Változók.....	36
5.19. Szoftvertesztelés.....	36
5.19.1. Meghajtó Címek Beolvasásának Tesztelése.....	36
5.19.2. Mappák és Fájlok Útvonalának Lekérdezése.....	37
5.19.3. Fájlok Olvasása és Írása.....	37
5.19.4. Manuális Tesztelés.....	37

6. Weboldal – Felhasználói Élmény és Arculati Koncepció.....	38
6.1.1 Kiemelt Értékek.....	38
6.1.2 Interakció és Navigáció.....	38
6.2. Frissítési Napló.....	39
6.2.1. Verziók Megjelenítése.....	39
6.2.2. Felhasználói Kommentek.....	39
6.2.3. Korábbi Verziók – Hibajavítások és Finomhangolások.....	39
6.2.4. Fejlesztési Irányok Áttekintése.....	40
6.3. Felhasználói Menü.....	40
6.3.1. Profil.....	40
6.3.2. Beállítások.....	40
6.3.3. Regisztráció.....	41
6.3.4. Bejelentkezés.....	41
6.3.5. Navigációs Logika.....	41
6.4. Rólunk.....	41
6.4.1. Fejlesztőcsapat.....	41
6.4.2. Általános Szerződési Feltételek.....	42
6.4.3. Jogi Megfelelés.....	42
7. Mobilalkalmazás.....	43
7.1.1 Működési Elv.....	43
7.1.2 Jogosultságok és Rendszerhozzáférés.....	43
7.1.3 Tárhelyinformációk Megjelenítése.....	43
7.1.4 Hálózati Kapcsolati Adatok.....	44
7.1.5 Háttérben Futó Szolgáltatás.....	44
7.2 MainActivity – A Mobilalkalmazás Működési Folyamata.....	44
7.2.1. Tárhelyfigyelés és Kijelzés.....	44
7.2.2. Helyi Hálózati IP-cím Lekérdezése.....	45
7.2.3. Automatikus Portkiosztás.....	45
7.2.4. Fájlszerver Indítása.....	45
7.2.5. Időzített Frissítési Mechanizmus.....	46
7.2.6. Háttérben Folyamatosan Futó Funkciók.....	46
7.3. FileServer – Beépített HTTP-alapú Fájlszerver.....	46
7.3.1. Gyökérkönyvtár Kezelése.....	47
7.3.2. Könyvtárlista Megjelenítése.....	47
7.3.3. Fájlok Letöltése.....	47
7.3.4. Hibakezelés.....	48
7.3.5. Kapcsolat a MainActivity-val.....	48

Fejlesztőcsapat

A fejlesztőcsapat három tagból áll, akik különböző szakterületeken biztosítják a rendszer működését és fejlődését.

- **Püspöki János Ákos – Csapatvezető/Fejlesztő** A projekt szakmai irányítója, a fájlkezelő program fő fejlesztője és a projekt koncepciójának megalkotója. Feladata a fejlesztési folyamat koordinálása és a rendszer funkcionális irányvonalának meghatározása.
- **Várhalmi Noel – Weboldal- és Adatbázis-fejlesztő** Feladata a webes felület megtervezése, működtetése és az oldal adatbázisának kialakítása. A webes rendszer stabilitása és átlátható működése nagyrészt az ő fejlesztői munkájára épül.
- **Bázing Bence – Mobilfejlesztő és Adatbázis-fejlesztő** A mobilos felületek kialakításáért és az adatbázis optimalizálásáért felel. Munkája biztosítja, hogy az OntroSYS mobilkörnyezetben is hatékonyan és stabilan működjön.

1.1. A projekt bemutatása

Az OntroSYS egy modern, keresztplatformos fájlkezelő szoftver, amelynek célja, hogy áthidalja a szakadékot a hagyományos fájlrendszer-kezelés és az internetes böngészési élmény között. A projekt olyan felhasználói igényekre ad választ, ahol a munkafolyamat során egyszerre szükséges a helyi állományok gyors elérése és a webes erőforrások hatékony használata.

A rendszer három fő komponensből áll:

- **Asztali alkalmazás** – a fájlkezelő program, amely a helyi állományok kezelését és a webes integrációt biztosítja.
- **Weboldal** – a projekt hivatalos felülete, ahol a felhasználók letölthetik a szoftvert, megtekinthetik a frissítési naplót, valamint kommentek formájában visszajelzést adhatnak a fejlesztőknek.
- **Mobilalkalmazás** – egy LAN-alapú kiegészítő modul, amely lehetővé teszi, hogy a telefon a főprogram számára hálózati „pendrive-ként” működjön, így vezeték nélküli fájlátvitelt biztosít.

Az OntroSYS így egy egységes ökoszisztémát alkot, amely a fájlkezelést, a webes funkcionalitást és a mobil eszközök integrációját egyetlen rendszerben egyesíti.

1.2. Célkitűzések

A szoftver fejlesztése során három fő irányelvet tartottunk szem előtt:

- **Felhasználóbarát megközelítés:** Letisztult, intuitív kezelőfelület, amely minimalizálja a tanulási görbét.
- **Keresztplatformos működés:** Teljes körű támogatás mind **Windows**, mind **Linux** alapú rendszerekre, biztosítva az azonos felhasználói élményt környezettől függetlenül.
- **Integrált böngészési élmény:** Egy teljes értékű, **Chromium** alapú böngésző közvetlen beágyazása a fájlkezelőbe, lehetővé téve a webes kutatást és fájlkezelést egyetlen alkalmazáson belül.

1.3. Főbb jellemzők

- Gyors és biztonságos fájlműveletek (másolás, mozgatás, törlés).
- Beépített Chromium motor a zavartalan webböngészéshez.
- Egységes kinézet és funkciók különböző operációs rendszereken.

1.4. A projekt története és kiválasztásának indokai

Az OntroSYS projekt alapötlete 2024 novemberében született meg, amikor a későbbi csapat alapítója, **Püspöki János Ákos** egy saját használatra szánt, egyszerű fájlkezelő alkalmazás fejlesztésébe kezdett. A korai verzió kizárólag alapvető funkciókat tartalmazott, és elsődleges célja a mindennapi fájlműveletek gyorsabbá és átláthatóbbá tétele volt.

2025 szeptemberére a kezdeti koncepció jelentősen kibővült. A projekt átalakítása és teljes újraírása ekkor vált vizsgaremekké, amely egyben a fejlesztőcsapat megalakulásának időpontja is. A korábbi egyszerű fájlkezelő helyét egy komplexebb, több komponensből álló rendszer vette át:

- a program Chromium-alapú böngészőmotort kapott a webes integrációhoz,
- elkészült a hivatalos weboldal letöltési és kommentelési lehetőségekkel,
- valamint megjelent a mobilalkalmazás, amely LAN-on keresztül „virtuális pendrive”-ként működik.

A projekt kiválasztását több szakmai és gyakorlati szempont indokolta:

- **Fejlesztői tapasztalat** A csapat tagjai korábban is foglalkoztak fájlkezelő rendszerek fejlesztésével, így a projekt jól illeszkedett a meglévő tudásbázishoz.

- **Átlátható, kétpaneles fájlkezelés** A cél egy olyan modern, kétablakos fájlkezelő létrehozása volt, amely nem terheli túl a felhasználót felesleges funkciókkal, mégis gyors és hatékony munkavégzést biztosít.
- **Keresztplatform kompatibilitás** A rendszer Java-alapú megközelítése lehetővé tette, hogy Windows és Linux környezetben is egységes funkcionalitást nyújtson.
- **Nyílt forráskódú, átlátható működés** A projekt egyik alapelve a transzparencia, amely mind a fejlesztési folyamatban, mind a szoftver működésében megjelenik.
- **Erőforrás-hatékonyság** A fejlesztők célja egy olyan alkalmazás létrehozása volt, amely nem pazarolja a rendszer erőforrásait, és alacsony terhelés mellett is stabilan működik.

Az OntroSYS így egy olyan projektté vált, amely egyszerre épít a korábbi tapasztalatokra, reagál a modern felhasználói igényekre, és hosszú távon is fenntartható, bővíthető alapot biztosít.

2. Rendszerterv és Architektúra

Az **OntroSYS** moduláris felépítésű, ahol a fájlrendszer-logika és a webes motor elkülönítve, mégis szoros egységben működik.

2.1. Technológiai Stack

- **Programozási nyelv:** Java (OpenJDK). A választás oka a natív keresztplatformos támogatás és a típusbiztos kódolás.
- **Grafikus felület:** Java Swing. Ez biztosítja az ablakkezelést és a panelek dinamikus váltását.
- **Böngésző motor:** JCEF (Java Chromium Embedded Framework).
 - *Egyedi módosítás:* A keretrendszer bizonyos részeit újraírtuk és optimalizáltuk, hogy az erőforrás-kezelés és a GUI-ba való beágyazás zökkenőmentes legyen az OntroSYS sajátos igényeihez.

2.2. Szerkezeti felépítés

A szoftver egy hibrid architektúrát követ, ahol a fájlkezelő motor és a Chromium folyamatok párhuzamosan futnak, biztosítva a felhasználói felület válasz készségét.

3. Funkcionális Specifikáció

Az OntroSYS a hatékonyságra és a multitaskingra épít.

3.1. Kétpaneles Fájlkezelés

A szoftver alapértelmezés szerint két függőleges panelre (ablakra) oszlik. Ez a klasszikus elrendezés teszi lehetővé a leggyorsabb műveletvégzést:

- **jobb kattintás alapú fájlfunkciók:** Fájlok kétirányú mozgatása/módosítása a bal/jobb oldali forráskönyvtárból a jobb/bal oldali forráskönyvtárba.
- **Gyors navigáció:** Mindkét panel független könyvtárfával rendelkezik.

3.2. Hibrid Web-Integráció

Az OntroSYS egyedülálló funkciója a panelek dinamikus átalakulása:

- **Konvertálható jobb panel:** Egyetlen gombnyomással a jobb oldali fájlböngésző egy teljes értékű Chromium alapú webböngészővé alakul át.
- **Kontextusfüggő munka:** Lehetővé teszi, hogy miközben a bal oldalon a helyi fájljainkat rendezzük, a jobb oldalon párhuzamosan töltsünk le adatokat, használjunk webes felhőalapú tárhelyet vagy kutatómunkát végezzünk.

3.3. Támogatott Platformok

A Java futtatókörnyezetnek köszönhetően a szoftver azonos funkciókkal bír:

- **Windows:** Teljes fájlrendszer-hozzáférés és rendszerspecifikus ikonok támogatása.
- **Linux:** Disztribúció-független működés (Debian, Fedora, Arch alapú rendszereken tesztelve).

4. Felhasználói Útmutató (Használat)

4.1. A kezelőfelület felépítése

Az **OntroSYS** indítás után egy kettéosztott ablakot jelenít meg:

- **Bal oldali panel:** Fix fájlrendszer-nézet.
- **Jobb oldali panel (Hibrid mód):** Alapértelmezésben fájlkezelő, de gombnyomásra teljes értékű **Chromium böngészővé** alakítható.

4.2. Fájlműveletek (Kontextusmenü)

A fájlok és mappák kezelése a jobb egérgombbal előhívható menün keresztül történik. Ez biztosítja a precíz vezérlést és megelőzi a fájlok véletlen elmozdítását.

Alapműveletek:

- **Másolás és Beillesztés:** Kattintson jobb gombbal a forrásfájlra, válassza a **Másolás** (Copy) opciót, majd a másik panel üres területére kattintva válassza a **Beillesztés** (Paste) parancsot.
- **Kivágás:** Ugyanaz a folyamat, mint a másolásnál, de a művelet végén a forrásfájl törlődik az eredeti helyéről.
- **Törlés:** A kijelölt elem végleges eltávolítása a jobb klikkes menüből.
- **Átnevezés:** Lehetővé teszi a kijelölt fájl vagy mappa nevének azonnali módosítását.

Szervezés és Karbantartás:

- **Mappa létrehozása:** Bármelyik panel területén jobb gombbal kattintva új könyvtárat hozhat létre a fájlok rendszerezéséhez.
- **Panel frissítése:** Amennyiben a fájlrendszer változásai (pl. külső letöltés) nem jelennének meg azonnal, a **Frissítés** opcióval kényszerítheti az adott ablak újratöltését.

4.3. A hibrid web-funkció használata

Amikor a jobb oldali panel böngésző módba vált, a jobb klikkes menü az internetes tartalomhoz igazodik (pl. hivatkozás megnyitása, kép mentése), kihasználva a **JCEF** által nyújtott natív Chromium képességeket.

4.4. Nyelvi beállítások és testreszabás

Az **OntroSYS** többnyelvű támogatással rendelkezik, amely a felhasználók által is bővíthető.

Nyelvváltás:

1. Kattintson a felső menüsorban található **Beállítások** fülre.
2. Válassza a **Nyelv** menüpontot. Ekkor a bal oldali panelben megjelenik az elérhető nyelvi fájlok listája.
3. Kattintson **duplán** a kívánt nyelvre.
4. A módosítás érvénybe léptetéséhez **indítsa újra** a szoftvert.

Saját nyelv hozzáadása (Lokalizáció):

A szoftver nyelvi fájljai egyszerű szöveges formátumot használnak, így bárki könnyen lefordíthatja a programot:

1. A **Beállítások / Nyelv** menüben a bal oldali ablakban kattintson jobb gombbal egy meglévő nyelvi fájlra (pl. english.anb_language).
2. Válassza a **Másolás**, majd a **Beillesztés** opciót.
3. **Nevezze át** a másolt fájlt az új nyelv nevére (pl. magyar.anb_language).
4. Nyissa meg a fájlt egy tetszőleges szövegszerkesztővel, és írja át a sorok tartalmát a megfelelő fordításra.
5. Mentés után a fájl azonnal megjelenik a választható nyelvek között.

4.5. Kinézet testreszabása (Look & Feel)

A Java Swing rugalmasságát kihasználva az OntroSYS lehetőséget ad a vizuális környezet megváltoztatására:

- **Elérése:** A felső menüsor **Look** fülére kattintva.
- **Beépített témák:** A szoftver **4 különböző előre definiált kinézettel** rendelkezik, amelyek között a felhasználó azonnal válthat a saját ízlésének megfelelően (pl. klasszikus Swing, natív rendszer-téma).

4.6. Hálózati kapcsolat (LAN / Mobil tárhely)

A felső menüsorban található **LAN** gomb segítségével a szoftver képes hálózati kapcsolatot létesíteni:

- **Célja:** Egyszerű csatlakozás mobil eszközök tárhelyéhez
- **Működése:** A csatlakozás után a távoli eszköz tartalma az egyik panelben tallózhatóvá válik, így a fájlok mozgatása a PC és a telefon között ugyanolyan egyszerűvé válik, mintha helyi meghajtók között dolgoznánk.

4.7. Beépített Szövegszerkesztő

Az OntroSYS tartalmaz egy beépített, pehelykönnyű szövegszerkesztő modult, amely feleslegessé teszi külső programok (pl. Notepad) megnyitását egyszerűbb feladatokhoz.

4.7.1 Funkciók és használat

- **Megnyitás:** Bármely .txt kiterjesztésű fájlra való **dupla kattintással** a szerkesztő azonnal megnyílik egy új ablakban.
- **Képességek:**
 - Szöveges tartalom megtekintése és valós idejű szerkesztése.
 - Új szöveges állományok létrehozása.
 - A fájl nevének módosítása közvetlenül a mentési folyamat során.
- **Hatékonyság:** Tudásában a Windows Jegyzettömbhöz (Notepad) hasonló, de szorosabban integrálódik az OntroSYS fájlkezelő logikájába, így a mentés után a fájllista azonnal frissül.

4.8. Hardverkövetelmények

Az **OntroSYS** nagy teljesítményű fájlkezelő és böngésző hibrid, amely a maximális sebesség érdekében közvetlen erőforrás-kezelést alkalmaz. A zavartalan működéshez az alábbi specifikációk betartása kötelező:

4.8.1 Tárolóegység (Storage)

- **Minimális szabad hely:** 4 GB.
- **Ajánlott szabad hely:** 10 GB (a Chromium motor gyorsítótárazása és a naplófájlok miatt).
- **Meghajtó típusa: Kizárólag SSD.**
 - *Indoklás:* A szoftver belső felépítése több mint 2000 állományt kezel, amelyeket a futtatás során véletlenszerűen (random access) hív meg. HDD használata esetén az elérési idő miatt a program válaszkészsége elfogadhatatlan mértékben romlik.

4.8.2 Memória (RAM)

- **Minimum követelmény:** 8 GB DDR3/DDR4/DDR5.
- **Memóriakezelés:** A program fix memóriefoglalási stratégiát alkalmaz. Az indításkor a rendszer-memória egy meghatározott szegmensét (jellemzően a 7 GB-os tartomány környékén) dedikáltan lefoglalja.
 - *Előny:* Ez a megoldás garantálja, hogy a szoftver futás közben nem lépi túl a számára kijelölt keretet, így nem okoz memóriaszivárgást vagy váratlan lassulást a többi alkalmazás számára. **8 GB RAM alatt a szoftver nem indul el.**

4.8.3 Processzor (CPU)

- **Magok száma:** Minimum 4 magos processzor ajánlott. A szoftver architektúrája erősen többszálú (multi-threaded), külön szálakon kezeli a fájlműveleteket, a GUI-t és a Chromium folyamatokat.
- **Specifikus követelmény:** Minimum **256 KB L1 Cache**.
 - *Műszaki háttér:* Az OntroSYS fix regiszter-helyeket használ az alacsony szintű műveletekhez. Ez teszi lehetővé a párhuzamosan futó **JDK 21** és **JDK 24** környezetek közötti stabil és gyors adatcserét.

4.9 Telepítés és Rendszerkövetelmények

Az **OntroSYS** telepítése egyszerűsített telepítővarázslóval történik, azonban a futtatáshoz elengedhetetlen a megfelelő környezet előzetes beállítása.

4.9.1 Rendszerkövetelmények

A szoftver zavartalan működéséhez a következő összetevőknek kell telepítve lenniük a rendszeren:

- **Operációs rendszer:** Windows 10/11 vagy modern Linux disztribúció.
 - **Java környezet:** * **JDK 21:** A stabil futtatókörnyezet alapjaihoz.
 - **JDK 24:** A legújabb funkciók és a JCEF optimalizált kiszolgálásához.
- Fontos:** Mindkét JDK verzió megléte kötelező a szoftver futtatásához!

4.9.2 Letöltési források

A programot két hivatalos csatornán keresztül érheti el:

1. **Elsődleges forrás:** akos0328.webredirect.org
2. **Alternatív forrás (Tüköroidal):** Amennyiben a weboldal nem elérhető, a legfrissebb verziók letölthetők a projekt **GitHub** oldalának "Releases" szekciójából.

4.9.3 Telepítési folyamat

1. Töltse le a platformjának megfelelő telepítőfájlt
2. Indítsa el a **Telepítő Varázslót**.
3. Kövesse az utasításokat, és adja meg a célkönyvtárat, ahová a programot telepíteni szeretné.
4. A telepítés befejezése után a szoftver az asztali ikonnal vagy a telepítési könyvtárból indítható.

5. Fejlesztői dokumentáció (OntroSYS Core)

5.1. Programindítási szekvencia (Bootstrapping)

Az alkalmazás belépési pontja az OntroSYS.java. Ez az osztály felelős a környezet inicializálásáért és a rendszerszintű komponensek megfelelő sorrendben történő meghívásáért.

5.1.1 Konfiguráció és Adatkezelés (FileIO.java, Gvar.java)

A folyamat a globális állapotok és a fájlrendszer-interfész előkészítésével indul:

- **Gvar.java (Global Variables):** A szoftver gerincét alkotó változók gyűjtőhelye. Itt public static módosítóval tároljuk a legfontosabb paramétereket, ami lehetővé teszi a rendszerszintű elérést és az osztályok közötti gyors adatcserét anélkül, hogy bonyolult objektumátadásokra lenne szükség.
- **FileIO.java (Nyelvi lokalizáció):** Az első futtatáskor vagy konfiguráció hiányában ez a modul felel az alapértelmezett környezet beállításáért.
 - **Fallback mechanizmus:** A rendszer alapértelmezetten az angol nyelvet keresi. Ha a felhasználó által beállított nyelvi fájl sérült vagy hiányzik, a szoftver automatikusan visszaáll az angol nyelvre a stabil működés érdekében.

5.1.2 Megjelenítési réteg (screen.java)

Miután az adatok rendelkezésre állnak, az OntroSYS.java meghívja a screen.java osztályt:

- Ez a modul inicializálja a fő **JFrame**-et.
- Felépíti a grafikus felület (GUI) vázát, meghatározza az ablakméreteket és elhelyezi a paneleket.

5.1.3 Interakciós réteg (actionlisteners.java, rightclickmenu.java)

A vizuális elemek után a logikai kapcsolatok jönnek létre:

- **actionlisteners.java:** Ez az osztály rendeli hozzá a funkciókat a GUI gombjaihoz és vezérlőihez. Minden kattintás és felhasználói beavatkozás ezen a központi eseménykezelőn fut keresztül.
- **rightclickmenu.java:** Külön kezelt modul a kontextusfüggő menükhöz. Tartalmazza a jobb egérgombos menü grafikai megjelenítését és a hozzájuk tartozó egyedi eseményfigyelőket (pl. másolás, törlés parancsok).

5.1.4 Adatbetöltés és Finalizálás (startupdataload)

A folyamat zárásaként lefut a startupdataload alprogram:

- **Kinézet (Look & Feel):** Beállítja a felhasználó által legutóbb választott vagy alapértelmezett vizuális témát.
- **Kezdeti tartalom:** Lefuttat egy frissítést a két fő ablakra, így a felhasználó azonnal a legutóbb megnyitott (vagy alapértelmezett) könyvtárak tartalmát látja, elkerülve az üres panelek megjelenését.

Műszaki megjegyzés

A Gvar.java használata lehetővé teszi a szoftver számára, hogy a szálak közötti kommunikáció és a különböző JDK-k közötti adatátvitel során egy biztos, fix pontra támaszkodhasson, minimalizálva az objektum-példányosításból adódó overheadet.

5.2. A grafikus felület inicializálása (screen.java)

A screen.java felelős a fő alkalmazásablak (Window) fizikai paramétereinek beállításáért és az alapvető konténer létrehozásáért.

- **Vázfelépítés:** Létrehozza a Gvar.frame objektumot (JFrame), amely az egész alkalmazás szülőkonformja.
- **Elrendezéskezelő:** A Gvar.panel egy **GridBagLayout**-ot kap. Ez a Java Swing egyik legösszetettebb, de legrugalmasabb elrendezéskezelője, amely lehetővé teszi, hogy a komponensek (menüsor, fájlablakok, böngésző) dinamikusan igazodjanak a képernyőmérethez.
- **Ablakparaméterek:** * Alapértelmezett méret: **1280x720**.
 - Minimum méret: **800x600** (ez garantálja, hogy a GUI elemek ne essenek szét túl kis ablakméret esetén).
 - A tartalom megjelenítését a GUI.java meghívása után a setContentPane parancs véglegesíti.

5.3. A GUI komponensek elhelyezése (GUI.java)

A GUI.java osztály nem csak létrehozza a vizuális elemeket, hanem a Gvar-ban tárolt globális kényszerek (GridBagConstraints) alapján rácsrendszerbe szervezi őket.

A GridBagLayout vezérlése:

A szoftver a következő logikai rácsbeállításokat használja a komponensek elosztásához:

- **weightx = 1:** A komponensek vízszintesen kitöltik a rendelkezésre álló helyet.
- **fill = GridBagConstraints.BOTH:** Biztosítja, hogy az elemek mind függőlegesen, mind vízszintesen nyúljanak, ha az ablakméret változik.
- **Struktúra:**
 1. **Felső menüsor:** A gridy = 0 (legfelső sor) pozícióban helyezkedik el. A gridwidth = 10 beállítás biztosítja, hogy a menüsor végigérjen az ablak tetején, függetlenül attól, hány oszlopra oszlik alatta a fájlkezelő rész.

Menürendszer és lokalizáció:

A GUI.java hívja meg a menubar.java-t is, amely beemeli a tényleges menüpontokat (Fájl, Beállítások, Look, LAN stb.). Itt történik a **nyelvi szövegek hozzárendelése** is: a program a Gvar-ban éppen aktív nyelvi térkép alapján feliratozza a gombokat és menüpontokat, így a felhasználó már a saját nyelvén látja a felületet az első pillanattól kezdve.

Műszaki megjegyzés:

Azzal, hogy a Gvar.gbc-t (GridBagConstraints) globális változóként van kezelve, elkerülendő az, hogy minden egyes metódusban új objektumot kelljen példányosítani. Ez egy memóriahatékony megoldás.

5.4. Dinamikus menürendszer felépítése (menubar.java)

A menubar.java osztály felelős az alkalmazás felső vezérlősorának (JMenuBar) strukturális felépítéséért és a többnyelvű feliratok inicializálásáért.

5.4.1. Struktúra és komponensek

A menürendszer három fő kategóriára oszlik, amelyeket a Gvar.menubar konténer fog össze:

- **Settings (Beállítások):** Tartalmazza a terminál, a nyelvváltó és az adatbázis-kezelő (DBeditor) elérését.
- **File (Fájl):** Itt található a hálózati (LAN) kapcsolatért felelős menüpont.
- **GUI (Look):** A vizuális témák (Metal, System, Nimbus, Motif) váltására szolgáló dedikált menü.

5.4.2. Dinamikus lokalizációs mechanizmus

A szoftver egyedi megoldást használ a feliratok kezelésére:

- Az osztály példányosítja a FileIO objektumot, amely közvetlen elérést biztosít a Gvar.nyelv változóban tárolt aktív nyelvi állományhoz.
- **Index-alapú beolvasás:** A nyelvinput.ReadLine(path, index) metódus hívásával a program sorszám szerint kéri le a fordításokat (pl. a 0. sor a "Settings", a 4. sor a "LAN" megfelelője).
- **Hatékonyság:** Ez a módszer minimalizálja a memóriaigényt, mivel nem tölti be a teljes nyelvi fájlt egyszerre, csak az éppen szükséges sorokat olvassa ki a komponensek feliratozásához.

5.4.3. Megjelenési stílusok (Look & Feel) kezelése

Míg a funkcionális menüpontok feliratai a nyelvi fájlból érkeznek, a GUI témák nevei (Metal, Nimbus stb.) fixen vannak kódolva. Ez logikus, hiszen ezek technikai elnevezések, amelyek nemzetközileg ismertek a Java Swing környezetben. Ezek a JMenuItem objektumok felelősek azért, hogy az `actionlisteners.java`-n keresztül aktiválják a UIManager stílusváltó parancsait.

5.5. Eseménykezelés és Interakciók (`actionlisteners.java`)

Az `actionlisteners.java` osztály központosítja a felhasználói beavatkozásokra adott válaszreakciókat. Ez a modul felelős a menüpontok, a fájllista interakciók és a Chromium motor összekapcsolásáért.

5.5.1. Globális komponensek és Böngésző inicializálás

Az osztály példányosítja az `OntroSYS_WEBTEST` objektumot, amely a **Chromium (JCEF)** alapú böngészőmotorért felel. Ez biztosítja, hogy a webes funkciók bármikor elérhetőek legyenek a fájlkezelő műveletek mellett.

5.5.2. Menüsor funkciók

A menubar gombjaihoz rendelt eseményfigyelők hívják meg a specifikus almodulokat:

- **Nyelvváltás:** Meghívja a `language_change` logikát, amely frissíti a globális nyelvi változókat.
- **Terminál és LAN:** Elindítja a beépített parancssori felületet vagy a hálózati csatlakozási varázslót.

5.5.3. Szinkronizált görgetési mechanizmus (`AdjustmentListener`)

Mivel a fájlok adatai (név, kiterjesztés, méret, módosítás dátuma) külön oszlopokban/listákban jelennek meg a jobb átláthatóság érdekében, a szoftver **szinkronizált görgetést** alkalmaz:

- Ha a felhasználó elmozdítja az egyik oszlop görgetősávját, a `syncScroll` figyelő azonnal átviszi az új pozíció értéket a többi oszlopnak is (`target.setValue`).
- Ez garantálja, hogy a fájl neve és a hozzá tartozó adatok mindig egy sorban maradjanak.

5.5.4. Intelligens duplikált kattintás (Double Click) logika

A fájllistában (namelist1) végzett dupla kattintás kontextusfüggő műveleteket vált ki:

- **Mappa esetén:** Frissíti a bal oldali útvonalat (Gvar.pathleft) és újratölti a könyvtárstruktúrát a dataprocess segítségével.
- **.anb_language fájl esetén:** Felismeri a saját nyelvi fájlformátumot, beállítja aktív nyelvnek, és a FileIO segítségével elmenti a választást a rendszerkonfigurációba (data.anb_syst).
- **.txt fájl esetén:** Automatikusan a beépített TextEditorv2-t indítja el.
- **Egyéb fájlok:** A Desktop.getDesktop().open(file) hívással átadja a fájlt az operációs rendszer alapértelmezett programjának.

Műszaki megjegyzés:

A szoftver **többszintű fallback** (visszalépési) rendszert használ: először megvizsgálja a saját belső kezelőit (nyelvi fájl, szövegszerkesztő), és csak ha nem talál egyezést, akkor fordul a natív operációs rendszerhez biztosítva a belső hibák kezelését és az alkalmazást Stabíl futását.

5.6. Adatfeldolgozási logika (dataprocess.java)

A dataprocess.java modul felelős a fájlrendszerből érkező adatok strukturálásáért, a panelek frissítéséért és a speciális rendezési algoritmusok futtatásáért. Az osztály szoros egységben dolgozik a fileolvasó és a drive segédosztályokkal.

5.6.1. Meghajtókezelés (updatedrive)

A szoftver indításakor és frissítéskor ez a metódus térképezi fel a számítógéphez csatlakoztatott logikai meghajtókat.

- **Működése:** A drive osztály segítségével lekéri a rendszertől az elérhető betűjeleket (pl. C:\, D:\), majd ezeket feltölti a Gvar.leftdrive és Gvar.rightdrive legördülő menübe (JComboBox).
- **Keresztplatformos hatás:** Windows alatt betűjeleket, Linux alatt a csatolási pontokat (mount points) kezeli egységes interfészen keresztül.

5.6.2. Panelstruktúra betöltése (loadleftsidestructure / loadrightsidestructure)

Ez a folyamat felelős az OntroSYS táblázatos megjelenítésének adatokkal való feltöltéséért. Mivel a szoftver külön listákban (JList) kezeli a fájlneveket, kiterjesztéseket, méreteket és dátumokat a szinkronizált görgetés miatt, az adatokat szét kell válogatni.

- **Adatszétválasztás:** A Gvar.leftdatastructureraw listában tárolt abszolút útvonalakat a metódus végigiterálja, és kinyeri belőlük:
 - **Név:** A fájl vagy könyvtár neve.
 - **Típus:** A kiterjesztés vagy a "Mappa" megjelölés.
 - **Dátum:** Az utolsó módosítás időpontja.
 - **Méret:** Az állomány fizikai mérete (bájtokban vagy formázva).
- **GUI frissítés:** Az elkészült ArrayList-eket tömbbé alakítva adja át a megfelelő namelist, extensionlist stb. komponenseknek.

5.6.3. Dinamikus újratöltés (reloadleftsidestructure)

Amikor a felhasználó mappát vált, ez a metódus vezényli le a frissítést:

1. Meghívja a fileolvasó osztályt az aktuális útvonalon (Gvar.pathleft).
2. Frissíti a nyers adatlistát.
3. Lefuttatja a fentebb említett load folyamatot a vizuális megjelenítéshez.

5.6.4. Intelligens rendezés (sortbynameleft)

Az OntroSYS egyik kiemelkedő technikai megoldása a **Natural Order Sort** (természetes rendezés) algoritmus implementálása.

- **A probléma:** A hagyományos ASCII rendezésnél a "file10.txt" megelőzi a "file2.txt"-t, ami zavaró a felhasználónak.
- **A megoldás (compareNatural):** * Az algoritmus karakterenként vizsgálja a neveket.
 - Ha számjegyeket talál, azokat nem karakterként, hanem teljes számértékként értelmezi és hasonlítja össze.
 - **Eredmény:** A listában a "file2.txt" után következik a "file10.txt", pontosan úgy, ahogy az emberi logika elvárja.
- **Kis- és nagybetű érzéketlenség:** A rendezés során a karaktereket kisbetűssé konvertálja az összehasonlítás idejére, így az "A" és "a" betűvel kezdődő fájlok egymás mellé kerülnek.

Műszaki megjegyzés

Architekturális megjegyzés: > A dataprocess osztály szétválasztja az adatgyűjtést (fileolvasó) az adatfeldolgozástól. Ez a modularitás teszi lehetővé, hogy a szoftver nagy méretű könyvtárak (több ezer fájl) esetén is stabil maradjon. A rendezési algoritmus közvetlenül a Gvar-ban tárolt nyers útvonal-listán (ArrayList<String>) operál, ami minimalizálja a memória-overheadet, mivel nem kell újabb és újabb objektum-példányokat létrehozni a rendezéshez, csupán a referenciák sorrendje változik. A System.out.println debug üzenetek a fejlesztői fázisban segítik a fájlrendszer-műveletek monitorozását, publikus kiadásnál ezek kikapcsolhatóak a naplózási szint módosításával.

5.7. Fájrendszer-beolvasó modul (fileolvaso.java)

A fileolvaso.java egy alacsony szintű segédosztály, amelynek egyetlen, de kritikus feladata van: egy megadott abszolút útvonalon található összes fájl és mappa fizikai elérési útjának begyűjtése.

5.7.1. Működési mechanizmus

A modul a filebeolvasas metóduson keresztül operál, amely egy String típusú abszolút útvonalat fogad paraméterként, és egy ArrayList<String> listával tér vissza.

1. **Objektum-példányosítás:** A metódus létrehoz egy java.io.File objektumot a megadott útvonal alapján.
2. **Iteráció:** A listFiles() metódus segítségével a szoftver lekéri a mappában található összes elemet egy File[] tömbbe.
3. **Adatkonverzió:** Egy for-each ciklus segítségével a program végighalad a talált elemeken, és mindegyiknek lekéri a teljes, abszolút elérési útját (getAbsolutePath()), majd ezt szöveges formátumban hozzáadja a visszatérési listához.

5.7.2. Hibakezelés és Naplózás

A fájlrendszer olvasása során számos hiba merülhet fel (pl. hozzáférés megtagadva, nem létező útvonal, sérült szektorok). Az OntroSYS ezt egy robusztus try-catch blokkal kezeli:

- **Konzolos visszajelzés:** A hibaüzenet megjelenik a fejlesztői konzolon a gyors hibakeresés érdekében.
- **Integrált Logolás:** A szoftver a hibát közvetlenül továbbítja a Terminal.Log felületére. Ez lehetővé teszi a felhasználó számára, hogy a programon belül is lássa, ha egy művelet (példányosítás vagy olvasás) meghiúsult, anélkül, hogy külső naplófájlokat kellene vizsgálnia.

Műszaki megjegyzés

Architektúrális megjegyzés: A fileolvaso osztály szándékosan lett minimalista módon megtervezve. Nem végez szűrést vagy formázást, csupán az abszolút elérési utakat gyűjti össze. Ez a megközelítés biztosítja a maximális sebességet az I/O műveletek során. Az abszolút útvonalak (Absolute File Path) használata garantálja, hogy a szoftver más moduljai (pl. a másolás vagy a szövegszerkesztő) mindig pontosan tudják, melyik fájlal kell dolgozniuk, elkerülve a relatív útvonalakból adódó félreértéseket. A Terminal.Log integráció biztosítja a transzparens működést, így a szoftver "öndiagnosztizáló" képességgel rendelkezik a fájlrendszer-hibák esetén.

5.8. Kontextusfüggő vezérlés (rightclickmenu.java)

A rightclickmenu.java osztály felelős a paneleken belül elérhető jobb klikkes (pop-up) menürendszer felépítéséért és a fájlműveletek logikai előkészítéséért.

5.8.1. Dinamikus menüfelépítés (setuprightclickmenu)

A menüpontok feliratainak betöltése itt is a lokalizációs motoron keresztül történik (FileIO.ReadLine), biztosítva a többnyelvűséget a felugró ablakokban is. Az osztály elkülöníti a bal és jobb oldali panelek eseményeit, de közös logikai alapokat használ.

5.8.2. Akció-kódolás és Állapotkezelés

A szoftver egy numerikus azonosító rendszert (Gvar.actions) használ a vágólap-műveletek megkülönböztetésére:

- **Másolás (1):** A kijelölt fájlok abszolút útvonalait a Gvar.copy listába menti, majd az állapotot 1-re állítja.
- **Kivágás (2):** Hasonló a másoláshoz, de az állapotkód 2 lesz, jelezve a későbbi áthelyezési szándékot.
- **Törlés (3):** Az állapotot 3-ra állítja, és azonnal meghívja a fileactionGUI-t a megerősítéshez és végrehajtáshoz.

5.8.3. Fájlműveletek Implementációja

Többszörös kijelölés kezelése

Az OntroSYS támogatja a csoportos fájlkezelést. Az `int[] selectedIndices = Gvar.namelist1.getSelectedIndices()` hívással a program lekéri az összes kijelölt elem indexét, majd egy ciklus segítségével kigyűjti azok abszolút útvonalait a `Gvar.leftdatastructureraw` listából.

Átnevezési mechanizmus

Az átnevezés a Java Swing `JOptionPane` dialógusát használja a felhasználói input bekérésére.

- A szoftver létrehoz egy új `File` objektumot a régi szülőkönyvtára és az új név alapján.
- A `renameTo()` metódus sikere esetén a program közvetlenül frissíti az adatstruktúrát (`Gvar.leftdatastructureraw.set()`), majd kényszerített frissítést (`reloadleftsidestructure`) küld a GUI-nak a szinkronitás megtartása érdekében.

Beillesztés és Végrehajtás

A paste funkció nem végez közvetlen I/O műveletet a menüosztályon belül. Ehelyett átadja a vezérlést a fileactionGUI-nak, amely a korábban beállított Gvar.actions és Gvar.copy tartalom alapján végzi el a fizikai másolást vagy mozgatást.

Műszaki megjegyzés

Architektúrális megjegyzés: A jobb klikkes menü implementációja szétválasztja a **szándékot** a **végrehajtástól**. A rightclickmenu csupán definiálja a feladatot (adatok kigyűjtése, művelet típusának meghatározása), míg a tényleges állomány-manipulációt a fileactionGUI osztály végzi el. Ez a megközelítés lehetővé teszi, hogy a szoftver a fájlműveletek alatt is válaszkész maradjon, és központosított felületet biztosítson a haladási sávok (progress bar) és hibaüzenetek megjelenítéséhez. Az isPopupTrigger() ellenőrzés a MouseAdapter-ben biztosítja a platformfüggetlen működést (Windows és Linux jobb klikk kezelése közötti különbségek áthidalása).

5.9. Fájlműveletek vizualizációja és aszinkron végrehajtása (fileactionGUI.java)

A fileactionGUI osztály egy speciális dialógusablakot (JDialog) biztosít, amely hidat képez a felhasználói felület és az alacsony szintű fájlmanipulációs osztályok (FileCopier, FileCut, Deletefile) között.

5.9.1. Aszinkron végrehajtás (Multithreading)

A szoftver egyik legfontosabb biztonsági funkciója, hogy a fájlműveleteket **külön szálon** (new Thread()) indítja el.

- **Műszaki előny:** Mivel a másolás vagy törlés nem a GUI-szálon (Event Dispatch Thread) fut, a főablak nem fagy le, a felhasználó tovább böngészhet vagy dolgozhat a másik panelen, miközben a háttérben zajlik a művelet.
- **Folyamat lezárása:** A szál végeztével a dialog.dispose() parancs automatikusan bezárja a folyamatjelző ablakot, majd a dataprocess segítségével frissíti az érintett paneleket.

5.9.2. Haladási naplózás és Sebességmérés

A szoftver valós idejű statisztikát készít a műveletről:

- **Fájlszámlálás (countRecursive):** A művelet megkezdése előtt a program rekurzív módon végigpásztázza a kijelölt mappákat, és megszámlolja az összes bennük lévő fájlt. Ez adja meg a JProgressBar maximum értékét.
- **UI Frissítési Időzítő (javax.swing.Timer):** 100 ezredmásodpercenként (0,1 mp) frissíti a dialógusablakot. Kiírja az aktuális sebességet (speedstring) és frissíti a haladási sáv értékét a már feldolgozott fájlok száma alapján.

5.9.3. Felhasználói kontroll (Pause/Stop)

A dialógusablak lehetőséget ad a folyamatok megszakítására vagy szüneteltetésére:

- **Pause (Szünet):** A newcop.stop logikai változó átbillentésével a másolási szál várakozási állapotba kényszeríthető.
- **Stop (Leállítás):** A cancel flag beállításával a művelet biztonságosan megszakad, a folyamatjelző pedig "Indeterminate" (végtelenített) módba vált, amíg a szál le nem áll.

Műszaki megjegyzés

Architektúrális megjegyzés: A fileactionGUI a **Strategy tervezési mintát** követi: a Gvar.actions értéke alapján dönti el, hogy melyik végrehajtó osztályt (FileCopier, FileCut vagy Deletefile) kell alkalmaznia, de a felhasználói felület és a haladáskövetés logikája egységes marad. A rekurzív fájlszámlálás biztosítja a pontos százalékos visszajelzést, míg a javax.swing.Timer alkalmazása megvédi a GUI-szálat a túlterheléstől, mivel nem engedi, hogy a másolási szál túl gyakran próbálja frissíteni a képernyőt.

5.10. Fizikai másolási mechanizmus (FileCopier.java)

A FileCopier osztály felelős a fájlok és mappák bit-szintű másolásáért. A modul úgy lett kialakítva, hogy szoros interakcióban álljon a fileactionGUI folyamatjelzőjével.

5.10.1. Rekurzív könyvtármásolás

A szoftver képes teljes mappastruktúrák kezelésére a copyDirectory metóduson keresztül:

- **Mappalétrehozás:** Ha a célhelyen nem létezik az adott könyvtár, a program létrehozza azt (mkdirs()).
- **Rekurzió:** A metódus önmagát hívja meg minden alkalommal, amikor egy almappát talál, így korlátlan mélységű könyvtárfát képes hűen átmásolni.

5.10.2. Intelligens fájlnev-ütközés kezelés

A copyFileWithSpeed metódus tartalmaz egy alapvető védelmi logikát a véletlen felülírás ellen:

- Ha a célhelyen már létezik egy azonos nevű fájl, a szoftver automatikusan egy _copy utótagot fűz az új állomány nevéhez, megőrizve ezzel mindkét verziót.

5.10.3. Folyamatvezérlés (Stop & Cancel)

A másolási szál folyamatosan figyeli a vezérlő flag-eket:

- **Szüneteltetés (stop):** Egy while(stop) ciklus segítségével a szál 200ms-os alvási periódusokba lép, amíg a felhasználó újra el nem indítja. Ez nem fogyaszt processzorerőforrást a várakozás alatt.
- **Megszakítás (cancel):** Minden fájl művelet előtt és a belső ciklusokban is ellenőrzi a szoftver a megszakítási szándékot. Ha aktív, a szál azonnal leáll és visszatér a már átmásolt fájlok számával.

5.10.4. Sebességmérés és Bufferelés

A hatékonyság és a vizualizáció érdekében a szoftver az alábbi technológiákat alkalmazza:

- **Buffered Streams:** A `FileInputStream` és `FileOutputStream` osztályokat `Buffered` réteggel látja el, és **8152 bájt** (8 KB) buffert használ. Ez az optimális méret, amely egyensúlyt teremt a memóriaigény és a lemezműveletek sebessége között.
- **Valós idejű kalkuláció:** A program méri az eltelt nanoszekundumokat (`System.nanoTime()`) és az átvitt bájtok mennyiségét. Ebből számítja ki a pillanatnyi sebességet.
- **Dinamikus formázás (`formatSpeed`):** A kiszámított bájtokat a szoftver automatikusan olvasható formátumba konvertálja (B/s, KB/s, MB/s vagy GB/s), amelyet a `speedstring` változón keresztül ad át a GUI-nak.

Műszaki megjegyzés

Implementációs részlet: A `FileCopier` aszinkronitásra lett tervezve. A belső adatátviteli ciklusban elhelyezett sebességmérő logika nem blokkolja a fájl írását, mivel egyszerű matematikai műveleteket végez. A 8 KB-os pufferek különösen SSD meghajtók esetén biztosítja az `OntroSYS` korábban említett hardverkövetelményeiben elvárt magas adatátviteli sebességet. A hibakezelés (`IOException`) biztosítja, hogy egyetlen fájl hibája (pl. olvasási hiba) ne döntse össze a teljes másolási folyamatot, hanem a hiba naplózásra kerüljön a `printStackTrace()` segítségével.

5.11. Fájlmozgatási mechanizmus (`FileCut.java`)

A `FileCut` osztály a "Kivágás és Beillesztés" (Move) funkcióért felelős. A szoftver kétlépcsős folyamatot alkalmaz: a fájlokat először a célhelyre másolja, majd validálás után törli az eredeti példányokat.

5.11.1. Atomi jellegű mozgatási logika

A biztonságos adatmozgatás érdekében a törlési fázis szigorúan a másolási fázis sikeres befejeződése után következik:

- **Fájlok esetén:** A program csak akkor hívja meg a `source.delete()` parancsot, ha a `copyFileWithSpeed` módszer hiba nélkül lefutott, és a felhasználó nem szakította meg a folyamatot (!cancel).
- **Könyvtárak esetén:** A teljes struktúra rekurzív másolása után a `deleteDirectory` módszer gondoskodik a forrásmappa maradéktalan eltávolításáról.

5.11.2. Biztonsági protokoll megszakítás (Cancel) esetén

A FileCut kiemelt figyelmet fordít az adatvesztés megelőzésére:

- Ha a felhasználó a folyamat közben a **Stop** gombra kattint, a cancel flag aktívvá válik.
- Ebben az esetben a program azonnal leállítja a másolást, és **nem hajtja végre a törlési fázist**. Így az adatok megmaradnak a forráshelyen, elkerülve, hogy egy félbehagyott mozgató miatt adatok vesszenek el.

5.11.3. Rekurzív forrástisztítás (deleteDirectory)

A mozgató végén a szoftvernek fel kell számolnia a már üres forráskönyvtárakat:

- A deleteDirectory metódus mélységi bejárással törli a fájlokat, majd alulról felfelé haladva távolítja el a már üres mappákat (dir.delete()).
- Ez garantálja, hogy ne maradjanak hátra "árva" mappaszerkezetek a merevlemezen.

Műszaki megjegyzés

Implementációs megjegyzés: A FileCut osztály szándékosan nem a Java natív Files.move() metódusát használja. Ennek oka a szoftver által kínált extra szolgáltatásokban rejlik: a natív mozgató nem teszi lehetővé a valós idejű sebességmérést, a folyamat szüneteltetését (Pause), illetve a 8 KB-os egyedi puffrelést. Ezzel a hibrid (Copy-then-Delete) megoldással az OntroSYS teljes kontrollt gyakorol az adatfolyam felett, és egységes vizuális visszajelzést (sebesség, progress bar) nyújt a felhasználónak minden művelet során.

5.12. Fájl-törlési mechanizmus (Deletefile.java)

A Deletefile osztály felelős az állományok és könyvtárstruktúrák végleges eltávolításáért. A modul a fileactionGUI aszinkron szálán fut, biztosítva a folyamatjelző sáv frissítését.

5.12.1. Rekurzív törlési protokoll

A Java natív File.delete() metódusa csak akkor képes törölni egy könyvtárat, ha az teljesen üres. Emiatt az OntroSYS egy mélységi bejárást alkalmaz:

- **deleteDirectory metódus:** A program először feltérképezi a mappa tartalmát.
- **Bejárás:** Ha fájlt talál, törli; ha alkönyvtárat, akkor rekurzívan meghívja önmagát.
- **Lezárás:** Csak miután minden belső elem törlésre került, távolítja el magát a szülőkönyvtárat.

5.12.2. Folyamatkövetés és Számlálás

A szoftver nem csupán elvégzi a törlést, hanem statisztikát is vezet:

- **deletedCount:** Minden egyes sikeresen törölt fájl vagy mappa után növeli a számlálót. Ez az érték szinkronban van a fileactionGUI progress bar-jával, így a felhasználó pontosan látja, hol tart a folyamat a korábban kiszámított (countFiles) összesített darabszámhoz képest.
- **Hibaellenőrzés:** A if (file.delete()) feltétel biztosítja, hogy a számláló csak akkor növekedjen, ha az operációs rendszer ténylegesen felszabadította az adott területet.

5.12.3. Biztonságos megszakítás

Hasonlóan a másolási műveletekhez, a törlés is bármikor megállítható:

- A fő ciklus minden iteráció után ellenőrzi a cancel flag állapotát.
- **Hatás:** Ha a felhasználó megszakítja a folyamatot, a törlés azonnal megáll. Ez kritikus fontosságú, ha a felhasználó rájön, hogy véletlenül jelölt ki egy fontos könyvtárat; a szoftver a lehető leggyorsabban reagál, hogy mentse a még nem érintett adatokat.

5.13. Képnézegető modul (Kísérleti fázis - imageviewer.java)

Az imageviewer osztály célja, hogy a felhasználóknak ne kelljen külső alkalmazást megnyitniuk a képi állományok ellenőrzéséhez. A modul különlegessége a szabad mozgatás és a görgetés alapú nagyítás támogatása.

5.13.1. Interaktív képmozgatás (huzogato)

A modul egy MouseAdapter segítségével implementálja a "drag-and-drop" jellegű navigációt a képen belül:

- **mousePressed:** Rögzíti a kattintás pontos koordinátáit a képernyőhöz viszonyítva.
- **mouseDragged:** Kiszámítja az elmozdulás mértékét az előző ponthoz képest, és folyamatosan frissíti az x és y koordinátákat. Ez lehetővé teszi a nagy felbontású képek "húzogatását" az ablakon belül.

5.13.2. Dinamikus skálázás (Zoom)

A nagyítási funkció a egérgörgőhöz van kötve:

- A mouseWheelMoved esemény módosítja az imgscale változót (20 egységes lépésközökkel).
- Ez az érték közvetlenül befolyásolja a kép szélességét és magasságát a renderelési ciklusban.

5.13.3. Valós idejű renderelési ciklus

A modul egy javax.swing.Timer-t használ (0 ms-os késleltetéssel), amely gyakorlatilag egy "game loop"-ként funkcionál:

- **Folyamatos frissítés:** A Timer minden ciklusban újraszámolja a kép méretét a getScaledInstance metódussal, figyelembe véve az aktuális zoom szintet (imgscale).
- **Képmínőség:** A skálázáshoz az Image.SCALE_FAST algoritmust használja a folyamatosság érdekében, hogy a nagyítás és mozgatás akadásmentes legyen.

Fejlesztési állapot: Az imageviewer modul jelenleg prototípus fázisban van, ezért nem része a publikus kiadásoknak. A modul demonstrálja a Swing keretrendszer határait a dinamikus képkezelés terén.

5.14. Dinamikus nyelvkezelési logika (language_change.java)

A language_change osztály felelős azért, hogy a szoftver belső erőforrásai közül elérhetővé tegye a nyelvi fájlokat a felhasználó számára. Ez a modul szorosan együttműködik a dataprocess osztállyal.

5.14.1. Útvonal-átirányítási mechanizmus

A modul a changelanguage metóduson keresztül egy átmeneti állapotot hoz létre a fájlkezelőben:

- **Állapotmentés (old_position):** A program elmenti a bal oldali panel aktuális útvonalát, hogy a nyelv kiválasztása után a felhasználó visszatérhessen az eredeti munkakörnyezetébe.
- **Rendszerkönyvtár megnyitása:** A szoftver a Gvar.pathleft értékét közvetlenül a ./src/main/java/SystemFiles (vagy a konfigurált nyelvi fájlok helye) útvonalra állítja.
- **Kényszerített frissítés:** A data.reloadleftsidestructure() meghívásával a bal oldali panel azonnal megjeleníti az elérhető .anb_language fájlokat, lehetővé téve a felhasználónak az új nyelv kiválasztását.

5.14.2. Adatfolyam-vezérlés

A modul nem csupán egy könyvtárváltó, hanem egy logikai kapu:

- Biztosítja, hogy a lokalizációs fájlok mindig elérhetőek legyenek a belső struktúrából.
- Statikus változókat használ (languagelist), hogy a kiválasztott nyelvi elem az egész alkalmazás szintjén referenciaként szolgálhasson a FileIO olvasó számára.

Műszaki megjegyzés

Architektúrális megjegyzés: A language_change osztály az OntroSYS azon filozófiáját tükrözi, miszerint a szoftver saját fájlkezelő képességeit használja fel az önmaga konfigurálására is. Ahelyett, hogy egy bonyolult, különálló beállító ablakot hoznánk létre a nyelvváltáshoz, a program a bal oldali panelt ideiglenesen egy "erőforrás-böngészővé" alakítja. Ez csökkenti a memóriahasználatot és egységes felhasználói élményt nyújt. Fontos megjegyezni, hogy az old_position változó tárolása kritikus, mivel ez biztosítja a szoftver állapotfolytonosságát (State Persistence) a konfigurációs művelet befejezése után.

5.15. Beépített Szövegszerkesztő Modul (TextEditorv2.java)

A TextEditorv2 egy JDialog alapú, aszinkron módon hívható szerkesztőfelület, amely natív rendszer megjelenéssel (System Look and Feel) rendelkezik.

5.15.1. Fájlkezelés és I/O műveletek

A szerkesztő a megnyitáskor azonnal beolvassa a cél fájl tartalmát:

- **Beolvasás:** FileReader és Scanner segítségével soronként olvassa be az adatokat, majd egy String pufferen keresztül tölti fel a JTextArea komponensbe.
- **Mentés:** A "Save" parancs hívásakor a FileWriter közvetlenül felülírja a fájlt a szövegdoz aktuális tartalmával. A rendszer robusztus hibakezeléssel rendelkezik, amely hiba esetén JOptionPane ablakban figyelmezteti a felhasználót.

5.15.2. Dinamikus Vizuális Beállítások

A korábbi verziókkal ellentétben a v2-es szerkesztő valós idejű kontrollt ad a megjelenítés felett:

- **Szövegméretezés:** A felhasználó a menüből növelheti vagy csökkentheti a betűméretet (Text Size +/-). A szoftver 4 egységenként módosítja a méretet, új Font objektumot példányosítva a folyamatosság érdekében.
- **Sor tördelése (Text Wrap):** Egy logikai kapcsoló (wraptext) segítségével a felhasználó válthat a vízszintes görgetés és az automatikus sortördelés között, ami különösen hasznos hosszú naplófájlok (logok) olvasásakor.

5.15.3. Speciális Fájl műveletek: Átnevezés

A szerkesztő egyik legfontosabb újítása a fájlok közvetlen átnevezésének lehetősége szerkesztés közben:

1. A program egy dedikált kis ablakot nyit, ahol a felhasználó megadhatja az új nevet.
2. Az **Enter** billentyű leütésekor (KeyAdapter) a szoftver kiszámítja az új abszolút útvonalat.
3. A renameTo() parancs után a szerkesztő frissíti a saját belső FilePATH változóját, így a mentés már az új néven történik tovább, anélkül, hogy a szerkesztőt be kellene zárni.

5.16. Fejlesztői Terminál és Rendszernapló (Terminal.java)

A Terminal osztály egy alacsony szintű diagnosztikai interfészt biztosít, amely lehetővé teszi a szoftver belső állapotának monitorozását és rejtett funkciók (például rendszerleállítás, kényszerített újraindítást, rendszer memória ürítést, statikus memória módosítást, RAM Dump, futtatási környezet változtatást, globális változók futás közbeni változtatását) elérését.

5.16.1. Központosított Naplózás (Static Log)

A modul legfontosabb eleme a public static JTextArea Log.

- **Műszaki előny:** Mivel statikus változóról van szó, a program bármely más osztálya (pl. TextEditorv2, FileCopier, FileIO) képes üzeneteket vagy hibaüzeneteket (Exception) küldeni ide anélkül, hogy példányosítania kellene a Terminált.
- **Görgethető felület:** A napló egy JScrollPane-be van ágyazva, így hosszú futási idő alatt keletkező tetemes mennyiségű hibaüzenet is visszakövethető marad.

5.16.2. Parancskezelő és Input Interfész

A terminál egy JTextField-en keresztül fogadja a parancsokat, melyeket egy ActionListener dolgoz fel az **Enter** billentyű leütésekor.

- **Parancs-memória:** A beírt parancsokat egy ArrayList<String> Memory objektumban tárolja (fejlesztési célú visszakereshetőséghez).
- **Vezérlő parancsok melyek elérhetők a felhasználói kiadásban:**
 - /help: Kilistázza az elérhető parancsokat a naplóba.
 - /clear: Kiüríti a napló tartalmát.
 - /terminate: Azonnali rendszerleállítás (System.exit(0)).
 - /exit: Csak a terminál ablakát zárja be (dispose()).

Műszaki megjegyzés

Diagnosztikai szerepkör: A Terminal az OntroSYS "fekete doboza". A legtöbb végfelhasználói verzióban a hozzáférés korlátozott, mivel olyan parancsokat tartalmaz, amelyek megkerülik a GUI biztonsági ellenőrzéseit (pl. kényszerített leállítás). A fejlesztés során ez a modul kritikus a hálózati kommunikáció és az aszinkron fájlműveletek hibakeresésében, mivel a System.out.println kimenetek mellett a GUI-n belül is láthatóvá teszi a háttérfolyamatok állapotát. Az elrendezés null layoutot használ a fix, ablakmérethez kötött pozicionálás érdekében.

5.17. Integrált Webes Motor (OntroSYS_WEBTEST)

Ez a modul a modern webes technológiákat emeli be az OntroSYS környezetébe. A JCEF használatával a szoftver egy teljes értékű, Chromium-alapú böngészőmotort kapott.

5.17.1. Windowless Rendering és Megjelenítés

A modul a CefSettings.windowless_rendering_enabled = true beállítást használja.

- **Célja:** Ez lehetővé teszi, hogy a böngésző ne egy külön ablakban, hanem a Swing GUI JPanel elemeként renderelődjön.
- **Elrendezés:** A felület GridBagLayout-ot használ, ahol a menubar (navigációs gombok és URL sáv) rögzített magasságú, míg a browserPanel kitölti a fennmaradó dinamikus teret.

5.17.2. Hibrid Görgetési Mechanizmus (JS Injection)

A modul egyik legkreatívabb megoldása a görgetés kezelése. Mivel a JCEF és a Swing közötti eseményátvitel néha instabil, a fejlesztők egy egyedi JavaScript injektálási eljárást alkalmaztak:

- **executeJavaScript:** Az egérgörgő eseményét a Java elkapja, kiszámítja a koordinátákat, majd egy beágyazott JS szkriptet futtat le a böngészőben.
- **Intelligens görgetés:** A szkript megkeresi a kurzor alatti legmélyebb görgethető elemet (elementFromPoint), és azon hajtja végre a scrollBy műveletet. Ez biztosítja, hogy ne csak az egész oldal, hanem a belső panelek is görgethetőek legyenek.

5.17.3. Fókuszkezelési Logika

A böngészőmotorok és a Java Swing ablakkezelője közötti fókuszharc elkerülése érdekében egy allowBrowserFocus flag került beépítésre:

- **Interakció:** A böngésző csak akkor kap fókuszt, ha a felhasználó ténylegesen rákattint a felületére.
- **Visszavétel:** Ha a böngésző elveszíti a fókuszt, a rendszer automatikusan visszaadja azt az URL sávnak, így biztosítva a folyamatos billentyűzet-bevitel lehetőségét.

5.17.4. Intelligens URL Kezelés

A szoftver tartalmaz egy automatikus protokoll-kiegészítőt:

- Ha a felhasználó csak egy címet ír be (pl. google.com), a rendszer a matches regex ellenőrzés után automatikusan eléfüzi a http:// előtagot, megelőzve ezzel a betöltési hibákat.

5.18. Globális Adatkezelés és Rendszerállapot (Gvar.java)

A Gvar osztály az OntroSYS architektúrájának tartóoszlopa. Ez a modul biztosítja a konzisztenciát a fájlkezelő bal és jobb oldali panelje, a hálózati réteg és a vizuális megjelenítés között.

5.18.1. GUI és Elrendezés Referenciák

A szoftver fő ablakának (frame) és a központi elrendezésért felelős GridBagConstraints-nek a tárolása itt történik. Ez teszi lehetővé, hogy például a Terminal vagy a TextEditorv2 tudja, melyik szülőablakhoz kell igazodnia (modalitás).

5.18.2. Kétpaneles Adatstruktúra

A fájlkezelő lelke a listák és scroll-panelek gyűjteménye. A Gvar külön-külön tárolja:

- **Megjelenítési réteg:** namelist, extensionlist, sizelist (a JList-ek, amiket a felhasználó lát).
- **Interakciós réteg:** namescrol, extensionscrol (a görgethetőségért felelős konténerek).
- **Adat réteg:** * leftdatastructureraw: A fájlok teljes, nyers elérési útja.
 - leftdatastructurerawsorted: A már szűrt/rendezett lista, ami a megjelenítés alapja.

5.18.3. Állapotmegőrzés (Navigation State)

A szoftver itt tárolja a pillanatnyi munkakörnyezetet:

- **pathleft / pathright:** A bal és jobb oldali panel aktuális könyvtárútvonala. Ezek módosításával navigálnak a modulok (pl. a language_change).
- **leftdrive / rightdrive:** A meghajtóválasztó comboboxok referenciái.
- **copy ArrayList:** Ez a "virtuális vágólap". Amikor a felhasználó kijelöl fájlokat másolásra vagy mozgatásra, azok elérési útjai ebbe a listába kerülnek, amíg a FileCopier vagy FileCut munkához nem lát.

5.18.4. Konfigurációs és Hálózati Változók

Bár a lanconnection még fejlesztés alatt áll, a Gvar már előkészített helyet biztosít a paramétereinek:

- **nyelv:** Az aktuálisan betöltött .anb_language fájl azonosítója.
- **ip / port:** A távoli kapcsolódáshoz szükséges hálózati végpontok adatai.
- **lookandfeelfnum:** A felhasználó által választott grafikai stílus (Look and Feel) indexe.

Műszaki megjegyzés

Architektúrális szempont: A **Gvar** használata lehetővé teszi az alacsony csatolást (**Low Coupling**) a funkcionális osztályok között. Például a **FileIO** osztálynak nem kell ismernie a **mainGUI** belső felépítését, elég, ha a **Gvar.pathleft** változót olvassa vagy írja.

5.19. Szoftvertesztelés

Az OntroSYS tesztelési folyamata a rendszer stabilitásának és megbízható működésének biztosítására épül. A program felépítése és funkcionális sajátosságai miatt az automatizálás csak korlátozottan alkalmazható, ezért a tesztelés jelentős része manuálisan, fejlesztői és béta tesztelői közreműködéssel történik. A kritikus funkciókhoz ugyanakkor célzott unit tesztek készültek.

5.19.1. Meghajtó Címek Beolvasásának Tesztelése

A meghajtók felismerése és listázása alapvető funkció, ezért külön unit teszt ellenőrzi:

- a rendszer által elérhető meghajtók helyes beolvasását,
- a meghajtók típusának (pl. helyi, külső, hálózati) megfelelő azonosítását,
- a hibakezelést nem elérhető vagy leválasztott meghajtók esetén.

A teszt célja annak biztosítása, hogy a fájlkezelő minden környezetben stabilan és pontosan jelenítse meg a rendelkezésre álló meghajtókat.

5.19.2. Mappák és Fájlok Útvonalának Lekérdezése

A második unit teszt a könyvtárstruktúra bejárásához kapcsolódik. Ellenőrzött elemek:

- mappák és fájlok útvonalának helyes összeállítása,
- relatív és abszolút elérési utak kezelése,
- hibakezelés nem létező vagy jogosultsággal nem rendelkező útvonalak esetén.

A teszt biztosítja, hogy a rendszer minden környezetben konzisztensen kezelje a fájlrendszer struktúráját.

5.19.3. Fájlok Olvasása és Írása

A harmadik unit teszt a fájlműveletek megbízhatóságát vizsgálja. A tesztelés kiterjed:

- fájlok megnyitására és tartalmuk beolvasására,
- új fájlok létrehozására és írására,
- meglévő fájlok módosítására,
- hibakezelésre (pl. írásvédett fájl, hiányzó jogosultság, sérült állomány).

A cél a stabil és kiszámítható fájlkezelési folyamatok garantálása.

5.19.4. Manuális Tesztelés

A program felépítése és a fájlkezelési műveletek sokfélesége miatt az automatizálás csak részben alkalmazható. A rendszer teljes körű ellenőrzése ezért manuálisan történik:

- a fejlesztők folyamatos belső tesztelése során,
- dedikált béta tesztelők bevonásával,
- valós használati környezetekben.

A manuális tesztelés lehetővé teszi a ritkán előforduló, összetett vagy környezetfüggő hibák azonosítását, amelyek automatizált eszközökkel nehezen reprodukálhatók.

Műszaki megjegyzés

Stabilitási figyelmeztetés: Az OntroSYS_WEBTEST jelenleg kísérleti (Experimental) státuszú. Mivel a JCEF natív (.dll / .so/ .openrt/ .opengl/ jdk21) könyvtárakat igényel a háttérben, a modul inicializálása (CefApp.startup) jelentős memóriát igényelhet. A fejlesztés jelenlegi fázisában a grafikai hibák elkerülése végett a GridBagLayout biztosítja a komponensek pontos illeszkedését. Ez a modul felelős az OntroSYS jövőbeli felhő-alapú integrációiért és a webes dokumentációk közvetlen eléréseért.

6. Weboldal – Felhasználói Élmény és Arculati Koncepció

A főoldal az OntroSYS vizuális és funkcionális identitásának központi eleme. A felület kialakításánál elsődleges szempont volt a letisztultság, a gyors értelmezhetőség és a professzionális megjelenés. A háttérben halványan megjelenő fájlkezelő ablak vizuális utalást ad a rendszer alapvető céljára: a fájlok hatékony, biztonságos és átlátható kezelésére. Ez a megoldás modern, informatív és azonnal érzékelteti a szoftver szakmai jellegét.

6.1.1 Kiemelt Értékek

A főoldalon megjelenő központi üzenet három alapvető tulajdonságot hangsúlyoz:

- **Gyorsaság** – optimalizált teljesítmény, minimális várakozási idő.
- **Biztonság** – háttérben futó ellenőrzések, stabil és kiszámítható fájlkezelési folyamatok.
- **Kényelem** – logikus felépítésű felhasználói felület, könnyen elérhető funkciók.

Ezek az értékek határozzák meg a rendszer működését és a felhasználói élményt.

6.1.2 Interakció és Navigáció

A főoldal egyik központi eleme a jól látható „**Letöltés**” gomb, amely egyetlen kattintással elindítja a telepítési folyamatot. A felső navigációs sáv egyszerű, mégis hatékony struktúrát követ:

- **Főoldal** – visszatérés a kezdőfelületre.
- **Frissítések** – verziótörténet, újdonságok és módosítások áttekintése.

A dizájn összességében professzionális, mégis felhasználóbarát, így a látogató már az első pillanatban egy megbízható és modern alkalmazással találkozik.

6.2. Frissítési Napló

A frissítési napló az OntroSYS fejlesztési folyamatának átlátható dokumentációja. A felület sötét témája hosszabb olvasás során is kényelmes, a verziók időrendben jelennek meg, így a felhasználó gyorsan áttekintheti a változásokat és a fejlesztési irányokat.

6.2.1. Verziók Megjelenítése

A verziók listázása időrendi sorrendben történik. A felület célja, hogy a felhasználó egyértelműen lássa:

- milyen módosítások történtek az egyes kiadásokban,
- mely funkciók kerültek bevezetésre vagy átdolgozásra,
- milyen hibajavítások járultak hozzá a stabilitáshoz.

A sötét témájú megjelenítés csökkenti a vizuális terhelést, így a hosszabb dokumentációk böngészése is komfortos.

6.2.2. Felhasználói Kommentek

A frissítési naplóhoz kapcsolódó kommentelési lehetőség közvetlen kommunikációs csatornát biztosít a fejlesztők és a felhasználók között. A funkció előnyei:

- **Visszajelzés valós használat alapján:** a felhasználók azonnal jelezhetik tapasztalataikat.
- **Fejlesztési irányok pontosítása:** a közösségi aktivitás segíti a prioritások meghatározását.
- **Rövid, célzott hozzászólások:** a példák alapján a felhasználók aktívan élnek a lehetőséggel.

6.2.3. Korábbi Verziók – Hibajavítások és Finomhangolások

A kisebb verziók, például az **1.1.1**, elsősorban hibajavításokat és stabilitási fejlesztéseket tartalmaznak. Jellemző elemek:

- kisebb funkcionális korrekciók,
- teljesítmény-finomhangolások,
- ritkán előforduló hibák javítása.

Ezek a kiadások rövidebb leírással rendelkeznek, de alapvető szerepet játszanak a rendszer megbízható működésében.

6.2.4. Fejlesztési Irányok Áttekintése

A frissítési napló nemcsak a múltbeli változásokat dokumentálja, hanem képet ad a fejlesztés hosszú távú irányairól is:

- a felület folyamatos modernizálása,
- a teljesítmény további optimalizálása,
- a felhasználói visszajelzések alapján történő funkcióbővítés.

6.3. Felhasználói Menü

A felhasználói menü a webes felület jobb felső sarkában található ikonról érhető el. A lenyíló struktúra öt elkülönített opciót tartalmaz, amelyek a fiókkezeléshez és az oldal információs funkcióihoz kapcsolódnak. A menü célja az egyszerű, gyors és logikus navigáció biztosítása.

6.3.1. Profil

A Profil szekció a felhasználó személyes adatainak és alapvető beállításainak kezelési felülete. A módosítható elemek:

- megjelenítési név,
- e-mail cím,
- egyéb alapadatok.

A felület átlátható, a változtatások azonnal elvégezhetők.

6.3.2. Beállítások

A Beállítások menüpont a rendszer működéséhez kapcsolódó konfigurációs lehetőségeket tartalmazza. Ide tartoznak:

- megjelenési témák,
- értesítési opciók,
- személyre szabható funkciók.

A cél, hogy a felhasználó a saját munkafolyamatához igazíthassa a rendszer viselkedését.

6.3.3. Regisztráció

A Regisztráció menüpont új felhasználói fiók létrehozását teszi lehetővé. A folyamat egyszerű, néhány alapadat megadásával aktiválható a fiók, amely ezt követően teljes hozzáférést biztosít a személyre szabott funkciókhoz.

6.3.4. Bejelentkezés

A Bejelentkezés gyors hozzáférést biztosít a már meglévő felhasználói adatokhoz és beállításokhoz. A funkció célja a zökkenőmentes belépés és a személyes munkakörnyezet azonnali elérése.

6.3.5. Navigációs Logika

A menü felépítése egyszerű és egyértelmű:

- minden funkció jól elkülönül,
- a legfontosabb opciók egy kattintással elérhetők,
- a felhasználó gyorsan megtalálja a keresett funkciót.

A kialakítás megfelel a modern webes felületek elvárásainak, támogatva a hatékony és intuitív használatot.

6.4. Rólunk

A „Rólunk” oldal az OntroSYS weboldalának információs központja, ahol a látogatók megismerhetik a fejlesztőcsapatot és a szoftver működéséhez kapcsolódó jogi hátteret. A felület letisztult, sötét témájú kialakítása illeszkedik az OntroSYS modern arculatához, és támogatja az áttekinthető információközlést.

6.4.1. Fejlesztőcsapat

A fejlesztőcsapat három tagból áll, akik különböző szakterületeken biztosítják a rendszer működését és fejlődését.

6.4.2. Általános Szerződési Feltételek

A „Rólunk” oldal külön szakaszban ismerteti az Általános Szerződési Feltételeket (ÁSZF). A dokumentáció kiemeli, hogy:

- a rendszer működése,
- a felhasználási feltételek,
- valamint a szerződéses keretek

összhangban állnak az Igazságügyi Minisztérium felhasználási típusú szerződéseire vonatkozó előírásokkal. Ez biztosítja, hogy a szoftver használata jogilag rendezett, átlátható és megfelel a hivatalos szabályozásoknak.

6.4.3. Jogi Megfelelés

A fejlesztők kiemelt figyelmet fordítanak a jogi megfelelésre. A hivatkozott szabályozások garantálják, hogy:

- a felhasználók jogai védve legyenek,
- a szoftver használata egyértelmű keretek között történjen,
- a fejlesztési folyamat megfeleljen a hivatalos előírásoknak.

7. Mobilalkalmazás

Az OntroSYS mobilalkalmazása kiegészítő modul, amely lehetővé teszi, hogy a telefon a helyi hálózaton keresztül a főprogram számára egy külső tárolóeszközként („virtuális pendrive-ként”) működjön. A rendszer célja a gyors, vezeték nélküli fájlátvitel biztosítása, valamint a mobil eszköz tárhelyének egyszerű elérése.

7.1.1 Működési Elv

A mobilalkalmazás LAN hálózaton keresztül kommunikál a számítógépen futó OntroSYS főprogrammal. A működés alapjai:

- a telefon háttérben futó szolgáltatása biztosítja a folyamatos elérhetőséget,
- a PC-s kliens a telefon tárhelyét úgy kezeli, mintha egy csatlakoztatott pendrive lenne,
- a fájlok és mappák átvitele kétirányú, késleltetés nélkül történik.

A megoldás előnye, hogy nincs szükség USB-kapcsolatra, a fájlkezelés teljes egészében vezeték nélkül valósul meg.

7.1.2 Jogosultságok és Rendszerhozzáférés

A mobilalkalmazás működéséhez szükséges:

- **hozzáférés az Android fájlrendszeréhez,**
- engedély a háttérben futó szolgáltatás működtetéséhez.

Ezek az engedélyek biztosítják, hogy a program képes legyen olvasni és írni a telefon tárhelyére, valamint folyamatos kapcsolatot tartson fenn a számítógéppel.

7.1.3 Tárhelyinformációk Megjelenítése

A felhasználói felület valós időben jeleníti meg:

- a telefon teljes tárhelykapacitását,
- az aktuálisan rendelkezésre álló szabad területet.

Ez lehetővé teszi, hogy a felhasználó azonnal lássa, mennyi adatot tud még átmásolni a mobil eszközre.

7.1.4 Hálózati Kapcsolati Adatok

A PC-s kliens csatlakozásához a mobilalkalmazás két fontos adatot jelenít meg:

- **LAN IP-cím** A telefon helyi hálózaton belüli címe, amelyre a számítógép csatlakozhat.
- **Portszám** A kommunikációhoz használt port, amelyen keresztül a főprogram eléri a mobilalkalmazást.

A két adat együtt biztosítja a stabil és biztonságos kapcsolat létrehozását.

7.1.5 Háttérben Futó Szolgáltatás

A mobilalkalmazás a háttérben fut, így:

- a kapcsolat folyamatos marad akkor is, ha a felhasználó más alkalmazást használ,
- a fájlátvitel megszakítás nélkül történhet,
- a rendszer minimális erőforrást használ, így nem terheli jelentősen az akkumulátort.

7.2 MainActivity – A Mobilalkalmazás Működési Folyamata

A mobilalkalmazás központi vezérlőeleme a *MainActivity*, amely a háttérben futó fájlszerver indításáért, a tárhelyadatok frissítéséért és a hálózati elérhetőség megjelenítéséért felel. A komponens folyamatosan figyeli a készülék állapotát, és 10 másodperces frissítési ciklusokban aktualizálja a kijelzett információkat.

7.2.1. Tárhelyfigyelés és Kijelzés

A rendszer valós időben monitorozza a telefon belső tárhelyének kihasználtságát. A működés fő elemei:

- a teljes, szabad és felhasznált tárhely kiszámítása,
- a kihasználtsági százalék megjelenítése,
- színkódolt állapotjelzés:
 - **zöld**: normál tartomány,
 - **sárga**: 75% feletti terhelés,
 - **piros**: 90% feletti telítettség.

7.2.2. Helyi Hálózati IP-cím Lekérdezése

A mobilalkalmazás automatikusan meghatározza a telefon LAN-on belüli IP-címét. A folyamat:

- a Wi-Fi adapter aktuális IP-címének lekérése,
- a cím átalakítása olvasható formátumra,
- a cím megjelenítése a felhasználói felületen.

Ez az IP-cím szükséges ahhoz, hogy a számítógépen futó OntroSYS kliens csatlakozni tudjon a mobil fájlserveréhez.

7.2.3. Automatikus Portkiosztás

A fájlserver indításakor a rendszer automatikusan választ egy szabad portot. A működés lépései:

- a szerver 0-ás portértékkal indul, amely automatikus kiosztást jelent,
- a rendszer lekérdezi a ténylegesen lefoglalt portot,
- a portszám megjelenik a felületen és az erre szolgáló beviteli mezőben.

A felhasználó így mindig pontosan látja, mely porton érhető el a mobil fájlserver.

7.2.4. Fájlserver Indítása

A mobilalkalmazás egy beépített fájlservert indít, amely a telefon tárhelyét teszi elérhetővé a PC számára. A szerver:

- a telefon külső tárhelyét (SD-kártya / belső megosztott tárhely) osztja meg,
- HTTP-alapú elérést biztosít,
- a háttérben fut, így más alkalmazások használata közben is aktív marad.

A szerver sikeres indítása után a felület kijelzi a teljes elérési útvonalat: **http://LAN_IP:PORT**

7.2.5. Időzített Frissítési Mechanizmus

A *MainActivity* 10 másodperces ciklusokban frissíti:

- a tárhelyadatokat,
- a hálózati IP-címet.

A frissítés csak akkor aktív, amikor az alkalmazás előtérben van, így energiatakarékos működést biztosít.

7.2.6. Háttérben Folyamatosan Futó Funkciók

Bár a felület csak előtérben frissül, a fájlserver:

- folyamatosan fut a háttérben,
- nem áll le akkor sem, ha a felhasználó más alkalmazásra vált,
- minimális erőforrást használ.

Ez biztosítja, hogy a PC-s OntroSYS kliens megszakítás nélkül tudjon csatlakozni a mobil eszközhöz.

7.3. FileServer – Beépített HTTP-alapú Fájlserver

A mobilalkalmazás egyik központi eleme a *FileServer* komponens, amely a telefon tárhelyét HTTP-alapú fájlserverként teszi elérhetővé a helyi hálózaton. A server a NanoHTTPD könyvtárra épül, és a PC-s OntroSYS kliens számára biztosítja a fájlok böngészését és letöltését.

7.3.1. Gyökérkönyvtár Kezelése

A szerver egy meghatározott gyökérmappát oszt meg (általában a telefon külső tárhelyét). A működés fő elemei:

- a gyökérkönyvtár tartalmának listázása,
- mappák és fájlok közvetlen elérése,
- a könyvtárstruktúra egyszerű, link-alapú megjelenítése.

A szerver minden beérkező HTTP-kérés esetén a gyökérmappán belül keresi a kért fájlt vagy mappát.

7.3.2. Könyvtárlista Megjelenítése

Ha a kliens a gyökérkönyvtárat vagy egy üres útvonalat kér le, a szerver HTML-formátumban listázza a mappa tartalmát. A lista:

- kattintható linkeket tartalmaz,
- minden elem közvetlenül elérhető,
- egyszerű, gyors böngészést tesz lehetővé.

Ez a megoldás biztosítja, hogy a PC-s OntroSYS kliens vagy akár egy böngésző is képes legyen a telefon tárhelyének áttekintésére.

7.3.3. Fájlok Letöltése

Ha a kliens egy konkrét fájlt kér le:

- a szerver megnyitja a fájlt,
- *chunked* HTTP-válaszban továbbítja a tartalmat,
- a MIME-típus alapértelmezés szerint *application/octet-stream*.

A chunked válasz lehetővé teszi nagy fájlok folyamatos, megszakítás nélküli továbbítását.

7.3.4. Hibakezelés

A szerver többféle hibát kezel:

- **nem létező fájl** → 404-es HTML válasz,
- **olvasási hiba** → rövid hibaüzenet,
- **érvénytelen útvonal** → alapértelmezett könyvtárlista.

A hibakezelés célja a stabil működés fenntartása még részben sérült vagy hiányos fájlrendszer esetén is.

7.3.5. Kapcsolat a MainActivity-val

A *FileServer* példányát a *MainActivity* indítja el, amely:

- automatikusan portot választ,
- megjeleníti a szerver elérési útvonalát,
- biztosítja a háttérben futó működést.

A két komponens együttműködése teszi lehetővé, hogy a telefon a PC számára egy hálózati „pendrive-ként” viselkedjen.